

Buenas prácticas de diseño de REST APIs

Ing. Rodolfo Christian Catunta Uturunco

¿Que significa REST?

- REST significa Representational State Transfer (Transferencia de Estado Representacional)
- Una API es REST si sigue los principios y estilo de diseño REST.
- Estos estilos fueron definidos en el año 2000 en la tesis doctoral del Dr. Roy Fielding.

Principio 1: Uniformidad de Interfaz

- Todas las peticiones a un recurso deben verse similares, no importando de donde viene la petición.
- El diseño debe satisfacer que el recurso accedido pertenezca a una única URI (Uniform Resource Identifier)
- Las URIs no deben ser muy largas, pero deben contener toda la información que un cliente puede necesitar.

Principio 2: Desacoplamiento Cliente / Servidor

- Las aplicaciones en el lado del cliente y el servidor deben ser independientes, la una de la otra.
- La única forma de que la aplicación del cliente interactúe con un recurso es a través de peticiones a una URI determinada que la aplicación del cliente conoce.
- De igual forma la aplicación en el servidor no puede cambiar la aplicación del cliente y se limita a responder a la petición HTTP con el recurso solicitado.

Principio 3: Ausencia de Estado

- Cada petición necesita colocar toda la información para ser procesada.
- En esencia esto hace que para usar una REST API no se necesite ninguna sesión en el lado del servidor. (No confundir con no usar seguridad en cada petición a través de token o contraseñas)
- La REST API no debería almacenar ningún dato de la petición del cliente (en la practica esto ya no ocurre, mayormente debido a las tecnologías de logging y análisis de datos).

Principio 4: Cacheabilidad

- Cuando sea posible los recursos mas consumidos pueden ser puestos en cache, ya sea en el servidor o en el cliente.
- Esto ayuda a mejorar el rendimiento en el lado del cliente.
- También a mejorar la escalabilidad en el lado del servidor.

Principio 5: Arquitectura de Sistema de Capas

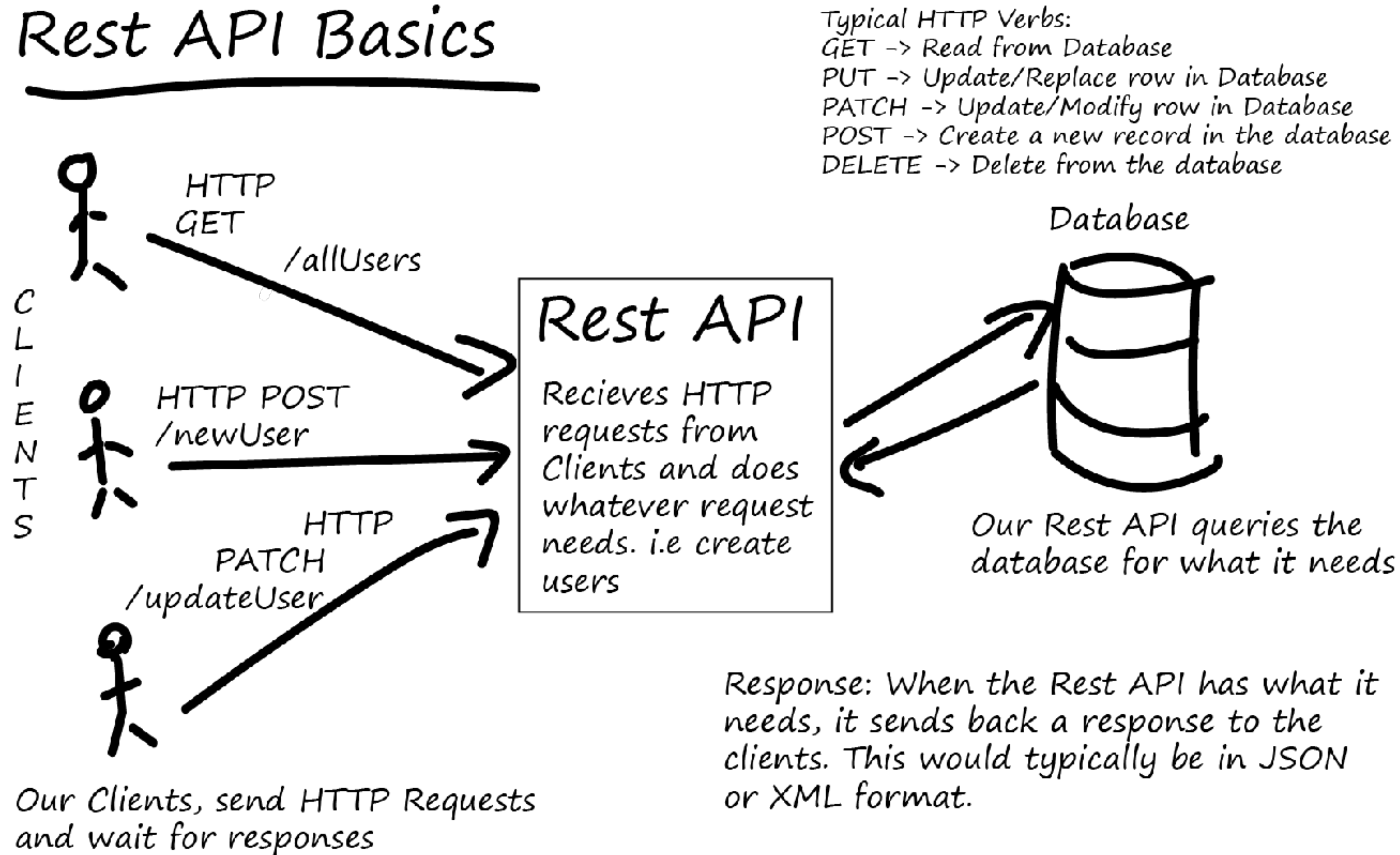
- En REST APIs las peticiones son manejadas por varias capas ya sean lógicas (en el código) o de infraestructura (proxy, firewalls, gateways, load balancers, etc).
- El cliente no debe ser expuesto a saber toda la ruta que sigue una petición hasta ser respondida (por temas de seguridad).

Principio 6: Código bajo demanda

- Se recomienda que las REST APIs devuelvan datos estáticos.
- También pueden devolver información acerca de otras URIs relativas a ese recurso.
- Sin embargo en ciertas ocasiones pueden devolver código ejecutable.
- Si es el caso el código solo debe correr bajo demanda del cliente.

¿Cómo funciona REST API?

Rest API Basics



¿Cómo funciona REST API?

Endpoint	<code>https://apiurl.com/review/new</code>
HTTP Method	<code>POST</code>
HTTP Headers	<code>content-type: application/json</code> <code>accept: application/json</code> <code>authorization: Basic abase64string</code>
Body	<pre>{ "review" : { "title" : "Great article!", "description" : "So easy to follow.", "rating" : 5 } }</pre>

Buenas prácticas

- Pensar en **recursos** no en acciones.
- En ese sentido las URIs deben reflejar sustantivos no verbos.
- Uso de plural para los recursos **students** y no student
- Uso de estados en la respuesta.
- Responder con JSON consistentes (misma estructura para datos o errores)
- Versionar la API.

Estados en la Respuesta

1XX	Códigos informativos	El servidor ha recibido la petición y procederá con ella.
2XX	Códigos de éxito	El servidor ha recibido, entendido y procesado la solicitud correctamente.
3XX	Códigos de redirección	El servidor ha recibido la solicitud, pero hay una redirección a alguna otra parte (o, en raras ocasiones, alguna acción adicional que debe completarse).
4XX	Códigos de error de cliente	El servidor no puede encontrar (o alcanzar) la página o la web. Se trata de un error del lado de la web.
5XX	Códigos de error de servidor	El cliente ha realizado una solicitud válida, pero el servidor ha fallado al completarla.

Estados en la Respuesta

Information

[100 - 199]

100 - Continue

101 - Switching Protocols

102 - Processing

103 - Early hints

Success [200 - 299]

200 - Ok

201 - Created

202 - Accepted

204 - No Content

206 - Partial Content

Redirect [300 - 399]

300 - Multiple choices

301 - Moved Permanently

304 - Not Modified

307 - Temporary redirect

308 - Permanent redirect

Client Error [400 - 499]

400 - Bad request

401 - Unauthorized

403 - Forbidden

404 - Not found

409 - Conflict

Server Error [500 - 599]

500 - Internal server error

501 - Not implemented

502 - Bad gateway

503 - Service unavailable

504 - Gateway timeout

Estados en la Respuesta

4XX Client Error			
400	Bad Request	502	Bad Gateway
401	Unauthorized	503	Service Unavailable
402	Payment Required	504	Gateway Timeout
403	Forbidden	505	HTTP Version Not Supported
404	Not Found	506	Variant Also Negotiates
405	Method Not Allowed	507	Insufficient Storage
406	Not Acceptable	508	Loop Detected
407	Proxy Authentication Required	510	Not Extended
408	Request Timeout	511	Network Authentication Required
		599	Network Connect Timeout Error

HTTP STATUS CODES

When a browser requests a service from a web server, an error may occur.
This is a list of HTTP status messages that might be returned.